

Cours ultra complet - BDD relationnelles, PostgreSQL, UML, SQL avance, vues, recursivite, PL/pgSQL

1. Objectif du cours

Ce support synthétise et explique l'ensemble des notions présentes dans les documents suivants :

- TD01 - modèle relationnel sous PostgreSQL
- TD01bis - requêtes SQL complexes
- TD02 - passage d'UML au modèle relationnel
- TD03 - passage UML -> relationnel niveau 2
- TD sur les vues
- TD05 - langage procédural PL/pgSQL
- TP02 - conception et implémentation de la base Distribill
- TP3 - récursivité, fermeture transitive, évolution du schéma
- TP4 - impact des choix de conception

L'idée n'est pas seulement de lister des commandes SQL. Le but est de comprendre :

- comment passer d'un diagramme UML à des tables relationnelles
- comment garantir l'intégrité des données
- comment écrire des requêtes simples puis avancées
- comment manipuler des hiérarchies avec SQL récursif
- comment utiliser les vues pour masquer, simplifier ou sécuriser l'accès aux données
- comment programmer avec PL/pgSQL
- comment faire des choix de conception qui ont un impact réel sur les performances et la qualité du schéma

Ce cours s'appuie aussi sur le schéma Distribill TP2 que tu as fourni.

2. Les bases du modèle relationnel

2.1 Une base relationnelle, c'est quoi ?

Une base de données relationnelle stocke les informations dans des relations, c'est-à-dire des tables.

Chaque table contient :

- des colonnes = les attributs

- des lignes = les n-uplets

Exemple :

```
```text
DEPT(nodept, nomdept, lieu)
```
```

représente une table des départements.

2.2 Vocabulaire fondamental

- Domaine : ensemble de valeurs possibles d'un attribut
- Attribut : colonne d'une relation
- N-uplet : ligne d'une relation
- Schéma relationnel : structure d'une relation
- Valeur d'une relation : ensemble des lignes présentes à un instant donné

2.3 Clé primaire, clé candidate, clé étrangère

- Clé candidate : ensemble minimal d'attributs permettant d'identifier une ligne
- Clé primaire : clé candidate choisie comme identifiant principal
- Clé étrangère : attribut ou groupe d'attributs pointant vers la clé primaire d'une autre table

Exemple :

```
```sql
create table dept(
nodept integer primary key,
nomdept varchar(20) not null,
lieu varchar(50) not null
);
```
```

Ici :

- `nodept` est la clé primaire
- `nomdept` et `lieu` sont obligatoires

2.4 Contraintes d'intégrité

Une bonne base de données ne stocke pas n'importe quoi. Elle impose des règles.

Principales contraintes :

- `PRIMARY KEY` : identifiant unique et non nul
- `NOT NULL` : valeur obligatoire
- `UNIQUE` : pas de doublon
- `CHECK` : contrainte métier locale
- `FOREIGN KEY` : respect des liens entre tables

Exemple :

```
```sql
check (sub <> sup)
```
```

interdit qu'un employé soit son propre supérieur.

2.5 Intégrité d'entité et intégrité référentielle

- Intégrité d'entité : une clé primaire ne peut pas être nulle
- Intégrité référentielle : une clé étrangère doit référencer une ligne existante

Conséquence :

si `emp.mondept` référence `dept.nodept`, on ne peut pas affecter un employé à un département inexistant.

3. Prise en main de PostgreSQL

3.1 Connexion

Dans les supports, la connexion se fait avec :

```
```bash
psql -U <login> -h servbdd pg_<login>
```
```

3.2 Commandes utiles dans `psql`

- `\d` : liste les tables
- `\d nom\_table` : décrit une table
- `\dn` : liste les schémas
- `\q` : quitter

- `\i fichier.sql` : exécuter un script

3.3 Schéma PostgreSQL

Un schéma est un espace de nommage logique.

Exemple :

```
```sql
create schema distribill1;
set schema 'distribill1';
```
```

Pourquoi c'est utile :

- ranger les objets
- éviter les collisions de noms
- gérer plusieurs versions d'une base

3.4 LDD vs LMD

- LDD = langage de définition des données
- ``create``, ``alter``, ``drop``
- LMD = langage de manipulation des données
- ``select``, ``insert``, ``update``, ``delete``

4. Traduire UML vers le modèle relationnel

Le passage UML -> relationnel est un point central de tes TD.

4.1 Une classe simple

Classe UML :

```
```text
DEPT(nodept, nomDept, lieu)
```
```

Traduction :

```
```sql
create table dept(
```

```

nodept integer primary key,
nomdept varchar(20) not null,
lieu varchar(50) not null
);
```

```

4.2 Association 1,N

Règle :

si une classe A est liée à une classe B avec cardinalité `1` du côté A et `N` du côté B, alors on place la clé primaire de A dans la table de B.

Exemple Distribill :

- un employé appartient à un département
- un département possède plusieurs employés

Donc :

```

```sql
mondept integer not null references dept(nodept)
```

```

dans `emp`.

4.3 Association N,N

Règle :

une association plusieurs-à-plusieurs devient une table d'association.

Exemple typique :

- un fournisseur fournit plusieurs produits
- un produit peut être fourni par plusieurs fournisseurs

Traduction :

```

```sql
create table fourniture(
nofour char(3),
noproduit char(3),

```

```

quantite integer not null,
primary key(nofour, noproduit),
foreign key(nofour) references fournisseur(nofour),
foreign key(noproduit) references produit(noproduit)
);
...

```

#### 4.4 Association porteuse d'attributs

Si l'association possède ses propres attributs, elle doit absolument devenir une table.

Exemple :

- `INSCRIPTIONS(date\_inscription)`
- `DOTATION(quantite\_dot, nomenclature)`

#### 4.5 Association réflexive

C'est une association d'une classe vers elle-même.

Exemple :

- un employé dirige d'autres employés

Traduction :

```

```sql
create table dirige(
sub integer primary key,
sup integer not null,
foreign key(sub) references emp(matr),
foreign key(sup) references emp(matr),
check (sub <> sup)
);
...

```

Lecture :

- `sub` = subordonné
- `sup` = supérieur
- `sub` est clé primaire : un employé ne peut avoir qu'un supérieur direct

4.6 Cardinalités et conséquences

- `0..1` : la clé étrangère peut être nulle
- `1..1` : clé étrangère obligatoire, éventuellement `unique`
- `1..N` : clé étrangère du côté N
- `N..N` : table d'association

4.7 Héritage UML

Dans TP3, le schéma évolue avec :

- `Personnel`
- `Employe`
- `President`

Quand on matérialise chaque classe de l'héritage par une relation, on est dans une stratégie "table par classe".

Exemple :

- `\_personnel(matr, nome, sal, dateemb, comm, mondept)`
- `\_employe(matr, poste)`
- `\_president(matr)`

`matr` est à la fois :

- clé primaire de chaque sous-table
- clé étrangère vers `\_personnel`

5. Étude complète du schéma Distribill TP2

Tu as fourni le script suivant comme base de travail. C'est le coeur du TP2.

5.1 Schéma logique

Entités :

- `dept` : départements
- `emp` : employés
- `dirige` : lien hiérarchique entre employés

5.2 Script d'implantation

``sql

drop schema if exists distribill1 cascade;

create schema distribill1;

set schema 'distribill1';

```
create table distribill1.dept(  
  nodept integer,  
  nomdept varchar(20) not null,  
  lieu varchar(50) not null,  
  constraint dept_pk primary key(nodept)  
);
```

```
create table distribill1.emp(  
  matr integer,  
  nomE varchar(20) not null,  
  sal numeric(10,2) not null,  
  poste varchar(20) not null,  
  dateEmb varchar(20) not null,  
  comm numeric(10,2) not null,  
  mondept integer not null,  
  constraint emp_pk primary key(matr)  
);
```

```
alter table distribill1.emp  
add constraint emp_fk_dept  
foreign key (mondept) references distribill1.dept(nodept);
```

```
create table distribill1.dirige(  
  sub integer,  
  sup integer not null,  
  constraint dirige_pk primary key(sub),  
  constraint dirige_fk_sub  
foreign key(sub) references distribill1.emp(matr),  
  constraint dirige_fk_sup  
foreign key(sup) references distribill1.emp(matr),  
  constraint dirige_sup_is_not_sub  
check (sub <> sup)
```



```
);  
...
```

5.3 Analyse conceptuelle du schéma

Table `dept`

Représente les départements de l'entreprise.

- `nodept` : identifiant du département
- `nomdept` : nom du département
- `lieu` : ville du département

Table `emp`

Représente les membres du personnel.

- `matr` : matricule
- `nomE` : nom
- `sal` : salaire
- `poste` : fonction
- `dateEmb` : date d'embauche
- `comm` : commission
- `mondept` : département d'appartenance

Table `dirige`

Représente la hiérarchie directe.

- un `sub` a exactement un supérieur direct
- un `sup` peut superviser plusieurs subordonnés

5.4 Graphe des contraintes d'intégrité référentielle

On peut le lire comme suit :

```
```text  
dept(nodept) <- emp(mondept)
emp(matr) <- dirige(sub)
emp(matr) <- dirige(sup)
...`
```

### 5.5 Remarque importante sur `dateEmb`

Dans un schéma propre, `dateEmb` devrait être de type `date`, pas `varchar(20)`.

Le plus propre serait :

```
```sql
dateEmb date not null
```
```

Pourquoi certains TP utilisent parfois du texte au départ :

- simplification de l'import
- éviter des erreurs de format lors du chargement CSV

Mais pour les requêtes de dates, le type `date` est nettement préférable.

## 5.6 Données du TP2

Départements :

- `10 FINANCES PARIS`
- `20 RECHERCHES GRENOBLE`
- `30 VENTES LYON`
- `40 FABRICATION ROUEN`

Exemples d'employés :

- `7839 LEROY PRESIDENT`
- `7566 MERCIER DIRECTEUR`
- `7698 NOIRET DIRECTEUR`
- `7788 DUBOIS INGENIEUR`
- `7902 CHATEL INGENIEUR`

## 5.7 Hiérarchie fournie

Tu as fourni les couples :

```
```sql
(7566,7839),
(7698,7839),
(7782,7839),
(7788,7566),
(7902,7566),
```

(7876,7788),
(7369,7902),
(7499,7698),
(7521,7698),
(7844,7698),
(7654,7698),
(7900,7698),
(7934,7782)
...

Interprétation :

- Mercier, Noiret et Lesage dépendent de Leroy
- Dubois et Chatel dépendent de Mercier
- Clement dépend de Dubois
- Leclerc dépend de Chatel

5.8 Exemple de peuplement minimal complet

```
```sql
insert into distribill1.dept(nodept, nomdept, lieu) values
(10, 'FINANCES', 'PARIS'),
(20, 'RECHERCHES', 'GRENOBLE'),
(30, 'VENTES', 'LYON'),
(40, 'FABRICATION', 'ROUEN');
```
```

Puis import CSV pour `emp`, puis insertions dans `dirige`.

5.9 Ce que le schéma garantit

- pas de doublon sur les départements
- pas de doublon sur les employés
- pas d'employé sans département
- pas de hiérarchie pointant vers un employé inexistant
- pas d'auto-subordination

5.10 Ce qu'il ne garantit pas encore

Le schéma ne garantit pas à lui seul :

- qu'il n'y a qu'un seul président
- qu'un directeur dirige uniquement les employés de son département
- qu'aucun cycle hiérarchique complexe n'existe
- qu'un département possède exactement un directeur

Ces règles relèvent soit :

- de requêtes de vérification
- de contraintes supplémentaires
- de triggers
- ou de logique applicative

6. Initialiser les données

6.1 Insertion manuelle

```
```sql
insert into dept values (10, 'FINANCES', 'PARIS');
```
```

6.2 Import CSV avec `\copy`

Dans `\psql` :

```
```sql
\copy distribill1.emp(matr, nome, sal, poste, dateemb, comm, mondept)
from 'employe.csv'
with (format csv, delimiter E'\t')
```
```

Attention :

- `\copy` est une commande client `\psql`
- elle ne s'exécute pas dans SQL Workbench/J comme une commande SQL standard

6.3 Import avec `WbImport`

Le TP montre aussi une importation via SQL Workbench/J :

```
```text
WbImport
```

```
-mode=insert
-file=employe.csv
-header=false
-table=emp
-schema='distribill1'
-fileColumns=matr,nome,sal,poste,dateemb,comm,mondept
-dateFormat='yyyy-MM-dd';
...
```

Idée :

- le fichier CSV fournit les lignes
- les colonnes du fichier sont mappées aux colonnes de la table

## 6.4 Script SQL

Pour `dirige`, l'initialisation se fait très bien avec une suite de `INSERT`.

## 7. Requêtes SQL fondamentales

### 7.1 Projection

Choisir certaines colonnes :

```
```sql
select nome, poste
from distribill1.emp;
...
```

7.2 Restriction

Filtrer des lignes :

```
```sql
select nome, poste
from distribill1.emp
where poste = 'INGENIEUR';
...
```

### 7.3 Tri

```
```sql
select nome, sal
from distribill1.emp
order by sal desc;
```
```

## 7.4 Suppression des doublons

```
```sql
select distinct poste
from distribill1.emp;
```
```

## 7.5 Jointures

Exemple employé + département :

```
```sql
select e.matr, e.nome, d.nomdept
from distribill1.emp e
join distribill1.dept d on d.nodept = e.mondept;
```
```

## 7.6 Jointure externe

Très importante pour afficher les départements même sans employé.

```
```sql
select d.nodept, d.nomdept, count(e.matr) as effectif
from distribill1.dept d
left join distribill1.emp e on e.mondept = d.nodept
group by d.nodept, d.nomdept;
```
```

# 8. Requêtes du TP2 sur Distribill : méthodes et exemples

## 8.1 Employés du département VENTES

```
```sql
select e.nome
```

```

from distribill1.emp e
join distribill1.dept d on d.nodept = e.mondept
where d.nomdept = 'VENTES';
'''

```

8.2 Directeur du département VENTES

```

```sql
select e.matr, e.nome
from distribill1.emp e
join distribill1.dept d on d.nodept = e.mondept
where d.nomdept = 'VENTES'
and e.poste = 'DIRECTEUR';
'''

```

## 8.3 Département créé mais sans personnel

Le sujet suggère le département sans ressources humaines.

```

```sql
select d.*
from distribill1.dept d
left join distribill1.emp e on e.mondept = d.nodept
where e.matr is null;
'''

```

8.4 Effectif de chaque département en activité

Un département "en activité" = avec au moins un employé.

```

```sql
select d.nodept, d.nomdept, count(*) as effectif
from distribill1.dept d
join distribill1.emp e on e.mondept = d.nodept
group by d.nodept, d.nomdept;
'''

```

## 8.5 Effectif de tous les départements

```

```sql
select d.nodept, d.nomdept, count(e.matr) as effectif

```

```
from distribill1.dept d
left join distribill1.emp e on e.mondept = d.nodept
group by d.nodept, d.nomdept;
---
```

8.6 Masse salariale globale

Avec commissions :

```
```sql
select sum(sal + comm) as masse_salariale
from distribill1.emp;

```

Sans commissions :

```
```sql
select sum(sal) as masse_salariale_hors_comm
from distribill1.emp;
---
```

8.7 Coût par département

```
```sql
select d.nomdept, sum(e.sal) as cout_hors_comm
from distribill1.dept d
join distribill1.emp e on e.mondept = d.nodept
group by d.nomdept;

```

## 8.8 Subalternes directs des directeurs

```
```sql
select sup.matr as matr_directeur, sup.nome as directeur,
sub.matr as matr_sub, sub.nome as subalterne
from distribill1.dirige di
join distribill1.emp sup on sup.matr = di.sup
join distribill1.emp sub on sub.matr = di.sub
where sup.poste = 'DIRECTEUR';
---
```


8.9 Ingénieurs subalternes directs des directeurs

```
```sql
select sub.matr, sub.nome
from distribill1.dirige di
join distribill1.emp sup on sup.matr = di.sup
join distribill1.emp sub on sub.matr = di.sub
where sup.poste = 'DIRECTEUR'
and sub.poste = 'INGENIEUR';
```
```

8.10 Successeur du directeur de RECHERCHES

Le sujet dit :

- prendre les plus proches collaborateurs hiérarchiques
- parmi eux, ceux qui sont ingénieurs
- retenir le plus ancien

```
```sql
select sub.matr, sub.nome, sub.dateemb
from distribill1.emp dir
join distribill1.dept d on d.nodept = dir.mondept
join distribill1.dirige di on di.sup = dir.matr
join distribill1.emp sub on sub.matr = di.sub
where d.nomdept = 'RECHERCHES'
and dir.poste = 'DIRECTEUR'
and sub.poste = 'INGENIEUR'
order by sub.dateemb asc
limit 1;
```
```

8.11 Vérifier qu'il n'y a qu'un seul président sans supérieur

```
```sql
select count(*) as nb_presidents_sans_sup
from distribill1.emp e
left join distribill1.dirige di on di.sub = e.matr
where di.sub is null;
```

...

L'idéal est d'obtenir `1`.

### 8.12 Mise à jour salariale

Augmenter RECHERCHES de 10% :

```
```sql
update distribill1.emp e
set sal = sal * 1.10
where e.mondept = (
select nodept
from distribill1.dept
where nomdept = 'RECHERCHES'
);
```
```

Corriger ensuite pour ne laisser qu'une hausse réelle de 2,5% :

Si les 10% ont déjà été appliqués, il faut revenir à la bonne valeur.

```
```sql
update distribill1.emp e
set sal = sal / 1.10 * 1.025
where e.mondept = (
select nodept
from distribill1.dept
where nomdept = 'RECHERCHES'
);
```
```

## 9. Requêtes SQL sur Fournisseur / Produit / Fourniture

Les TD01 et TD01bis portent beaucoup sur cette base.

### 9.1 Structure logique

- `fournisseur(nofour, nomf, ville)`
- `produit(noproduit, nomp, origine, couleur)`

- `fourniture(nofour, noproduit, quantite)`

## 9.2 Requêtes de base

Produits originaires de Dijon :

```
```sql
select *
from produit
where origine = 'Dijon';
```
```

Numéros des fournisseurs qui fournissent quelque chose :

```
```sql
select distinct nofour
from fourniture;
```
```

Numéros des fournisseurs qui ne fournissent rien :

```
```sql
select f.nofour
from fournisseur f
left join fourniture fo on fo.nofour = f.nofour
where fo.nofour is null;
```
```

## 9.3 Logique "au moins", "pas", "seulement"

Fournisseurs qui fournissent `P6` :

```
```sql
select distinct nofour
from fourniture
where noproduit = 'P6';
```
```

Fournisseurs qui ne fournissent pas `P6` :

```
```sql
select nofour
```

```

from fournisseur
where nofour not in (
select nofour
from fourniture
where noproduit = 'P6'
);
```

```

Fournisseurs qui fournissent quelque chose mais pas `P6` :

```

```sql
select distinct nofour
from fourniture
where nofour not in (
select nofour
from fourniture
where noproduit = 'P6'
);
```

```

Fournisseurs qui ne fournissent que `P6` :

```

```sql
select nofour
from fourniture
group by nofour
having bool_and(noproduit = 'P6');
```

```

## 9.4 Requêtes avec auto-jointure

Produits de même couleur :

```

```sql
select p1.noproduit, p2.noproduit
from produit p1
join produit p2
on p1.couleur = p2.couleur
and p1.noproduit < p2.noproduit;
```

```

...

## 9.5 Requêtes d'agrégation

Nombre de produits différents fournis par fournisseur :

```
```sql
select nofour, count(distinct noproduit) as variete
from fourniture
group by nofour;
```
```

Quantité totale fournie par fournisseur :

```
```sql
select nofour, sum(quantite) as qte_totale
from fourniture
group by nofour;
```
```

## 9.6 Résultats calculés sur les données du TD01bis

Variété et quantité globale :

- `F1 -> 4 produits, quantité totale 12`
- `F2 -> 2 produits, quantité totale 2`
- `F3 -> 2 produits, quantité totale 6`
- `F4 -> 3 produits, quantité totale 12`
- `F5 -> 1 produit, quantité totale 10`

Produits verts :

- `F1 -> 2 produits verts, quantité 10`
- `F4 -> 2 produits verts, quantité 10`
- `F5 -> 1 produit vert, quantité 10`

Fournisseurs ayant fourni plus de 2 produits différents :

- `F1`
- `F4`

## 9.7 `GROUP BY` et `HAVING`

`WHERE` filtre avant regroupement.

`HAVING` filtre après regroupement.

Exemple :

```
```sql
select nofour
from fourniture
group by nofour
having count(distinct noproduit) > 2;
```
```

## 9.8 Division relationnelle

Sujet classique :

"Quels sont les fournisseurs qui fournissent tous les produits originaires de Dijon ?"

Produits de Dijon :

- `P1`
- `P4`
- `P6`

Écriture robuste :

```
```sql
select f.nofour
from fournisseur f
where not exists (
select 1
from produit p
where p.origine = 'Dijon'
and not exists (
select 1
from fourniture fo
where fo.nofour = f.nofour
and fo.noproduit = p.noproduit
)
);
```
```

...

Idée :

- on cherche les fournisseurs pour lesquels il n'existe aucun produit de Dijon manquant

### 9.9 "Exactement les mêmes produits"

Problème très classique et très formateur.

Deux fournisseurs `f1` et `f2` fournissent exactement les mêmes produits si :

- tout produit fourni par `f1` est fourni par `f2`
- et tout produit fourni par `f2` est fourni par `f1`

### 9.10 "Aucun produit en commun"

Deux fournisseurs n'ont aucun produit commun si l'intersection de leurs ensembles de produits est vide.

## 10. Requêtes complexes : méthodes générales

Quand un énoncé devient difficile, il faut le traduire en logique.

### 10.1 Repérer le type de problème

Question typique -> stratégie :

- "combien" -> `count`
- "somme" -> `sum`
- "moyenne" -> `avg`
- "max/min" -> `max`, `min`
- "au moins un" -> `exists`
- "aucun" -> `not exists`
- "tous" -> double négation, division relationnelle
- "même couleur", "même produit" -> auto-jointure
- "hiérarchie" -> requête récursive

### 10.2 Toujours raisonner en étapes

Exemple :

"les fournisseurs qui fournissent la plus grande quantité cumulée de produits verts"

Étapes :

1. garder les fournitures de produits verts
2. sommer par fournisseur
3. comparer à la somme maximale

### 10.3 Sous-requêtes scalaires

```
```sql
select *
from emp
where sal = (select max(sal) from emp);
```
```

### 10.4 Sous-requêtes corrélées

```
```sql
select f.nofour
from fournisseur f
where exists (
select 1
from fourniture fo
where fo.nofour = f.nofour
);
```
```

### 10.5 CTE avec `WITH`

Très utile pour découper la logique.

```
```sql
with verts as (
select fo.nofour, fo.quantite
from fourniture fo
join produit p on p.noproduit = fo.noproduit
where p.couleur = 'Vert'
)
select nofour, sum(quantite)
from verts
```



```
group by nofour;
```

```
---
```

11. Vues relationnelles

Les vues sont au centre du TD-Vues et de TP3.

11.1 Définition

Une vue est une table virtuelle définie par une requête.

```
```sql
```

```
create view LannionProducteurs as
```

```
select *
```

```
from Producteur
```

```
where ville = 'Lannion';
```

```

```

### 11.2 Intérêt des vues

- simplifier des requêtes compliquées
- masquer certaines colonnes
- restreindre les lignes visibles
- stabiliser l'interface logique d'une base
- exposer une structure plus pratique que les tables physiques

### 11.3 Interroger une vue

```
```sql
```

```
select nom
```

```
from LannionProducteurs;
```

```
---
```

ou

```
```sql
```

```
select *
```

```
from LannionProducteurs
```

```
where nom like 'S%';
```

```

```

## 11.4 Vue et table de base

Interroger une vue revient à réutiliser sa requête de définition.

Exemple équivalent :

```
```sql
select nom
from Producteur
where ville = 'Lannion';
```
```

## 11.5 Vue modifiable

Certaines vues peuvent accepter `insert`, `update`, `delete`.

En pratique, plus une vue est simple, plus elle a de chances d'être modifiable.

Souvent modifiables :

- vue sur une seule table
- sans agrégation
- sans `group by`
- sans `distinct`
- sans jointure complexe

## 11.6 Pourquoi un `insert` peut "disparaître" de la vue

Si on insère dans la vue `LannionProducteurs` un producteur situé à Rennes :

- l'insertion peut passer dans la table de base
- mais la ligne n'apparaît pas dans la vue, car elle ne satisfait pas `ville = 'Lannion'`

## 11.7 `WITH CHECK OPTION`

Cette clause empêche d'insérer ou de modifier une ligne qui sortirait du prédicat de la vue.

```
```sql
create view LannionProducteurs as
select *
from Producteur
where ville = 'Lannion'
with check option;
```

...

Conséquence :

- impossible d'insérer un producteur à Rennes via cette vue

11.8 Vues par projection

Exemples du TD :

```
```sql
```

```
create view NomsConsommateurs as
select nom, prenom
from Consommateur;
```

```
create view ListeProduits as
select id, description
from Produit;
```

...

Problème :

si la vue n'inclut pas la clé primaire ou des colonnes obligatoires, certaines mises à jour deviennent impossibles ou ambiguës.

### 11.9 Vues non modifiables

En général, les vues suivantes ne sont pas modifiables au sens simple :

- vues avec agrégats
- vues avec `group by`
- vues avec `distinct`
- vues avec certaines jointures

Exemple :

```
```sql
```

```
create view NbProduitsParProducteur as
select id_producteur, count(*) as nb
from Produit
group by id_producteur;
```

...

On ne sait pas "dans quelle ligne de base" traduire un `insert` dans cette vue.

11.10 Vues utiles du TD

Produits produits et consommés dans la même ville :

idée :

- relier producteur, produit, consommateur, consommation
- comparer les villes

Produits non consommés :

```
```sql
create view ProduitsNonConsommes as
select p.*
from Produit p
left join Consommation c on c.id_produit = p.id
where c.id_produit is null;
```
```

Nombre de produits par producteur :

```
```sql
create view NbProduitsParProducteur as
select pr.id, pr.nom, count(p.id) as nb_produits
from Producteur pr
left join Produit p on p.id_producteur = pr.id
group by pr.id, pr.nom;
```
```

12. Hiérarchie, récursivité et fermeture transitive

Le TP3 introduit une notion fondamentale : les requêtes récursives.

12.1 Subalterne direct

```
```sql
select sub.matr, sub.nome
from distribill1.dirige d
join distribill1.emp sup on sup.matr = d.sup
```

```
join distribill1.emp sub on sub.matr = d.sub
where sup.nome = 'MERCIER';

```

Résultat calculé sur tes données :

- `7788 DUBOIS`
- `7902 CHATEL`

## 12.2 Deux niveaux de hiérarchie

Sans récursivité générale, on peut faire une requête à deux niveaux :

```
```sql
select distinct e2.matr, e2.nome
from distribill1.emp chef
join distribill1.dirige d1 on d1.sup = chef.matr
join distribill1.emp e1 on e1.matr = d1.sub
left join distribill1.dirige d2 on d2.sup = e1.matr
left join distribill1.emp e2 on e2.matr = d2.sub
where chef.nome = 'MERCIER';
---
```

Mais cette approche ne passe pas à l'échelle.

12.3 Requête récursive avec `WITH RECURSIVE`

```
```sql
with recursive hierarchie as (
select d.sup, d.sub, 1 as niveau
from distribill1.dirige d

union all

select h.sup, d.sub, h.niveau + 1
from hierarchie h
join distribill1.dirige d on d.sup = h.sub
)
select e.matr, e.nome, h.niveau
from hierarchie h
join distribill1.emp chef on chef.matr = h.sup
```

```

join distribill1.emp e on e.matr = h.sub
where chef.nome = 'MERCIER'
order by h.niveau, e.matr;
'''

```

## 12.4 Lecture de la requête récursive

Partie initiale :

- récupère les arcs directs `sup -> sub`

Partie récursive :

- si `A` dirige `B`
- et `B` dirige `C`
- alors `A` dirige aussi `C` à un niveau plus profond

## 12.5 Résultat pour Mercier

Subalternes de Mercier :

- niveau 1 : `7788 DUBOIS`, `7902 CHATEL`
- niveau 2 : `7876 CLEMENT`, `7369 LECLERC`

## 12.6 Fermeture transitive

La fermeture transitive d'une relation hiérarchique contient tous les couples :

- supérieur
- subordonné atteignable

quel que soit le nombre de niveaux.

Exemple :

si `A -> B` et `B -> C`, alors la fermeture transitive contient aussi `A -> C`.

## 12.7 Nombre de subordonnés

Sur tes données :

- Leroy : `13`
- Mercier : `4`
- Noiret : `5`

- Dubois : `1`

## 12.8 Pièges des requêtes récursives

- risque de cycles
- doublons si on ne fait pas attention
- nécessité de bien choisir `union` ou `union all`

## 13. Évolution du schéma avec héritage et vues de compatibilité

Dans TP3, on demande d'évoluer vers `distribill2`.

### 13.1 Nouveau modèle conceptuel

On distingue :

- `Personnel`
- `Employe`
- `President`

et on introduit une propriété calculée :

```
```text
prime_enc = (sal * 5/100) * NbSub
```
```

### 13.2 Pourquoi séparer `Personnel`, `Employe`, `President` ?

Parce qu'en UML, l'héritage exprime :

- des propriétés communes
- des spécialisations

Ici :

- tout le monde a `matr`, `nomE`, `sal`, `dateEmb`, `comm`
- seuls les employés "ordinaires" ont `poste`
- le président forme une spécialisation particulière

### 13.3 Implantation possible

```
```sql
create table distribill2._personnel(
```

```

matr integer primary key,
nome varchar(20) not null,
sal numeric(10,2) not null,
dateemb date not null,
comm numeric(10,2) not null,
mondept integer not null references distribill2.dept(nodept)
);

create table distribill2._employe(
matr integer primary key references distribill2._personnel(matr),
poste varchar(20) not null
);

create table distribill2._president(
matr integer primary key references distribill2._personnel(matr)
);
...

```

13.4 Garder `SELECT \* FROM EMP` fonctionnel

Le sujet demande de conserver les anciennes requêtes.

Solution élégante :

créer une vue `emp` qui recompose les données.

```

```sql
create view distribill2.emp as
select p.matr, p.nome, p.sal,
e.poste,
p.dateemb, p.comm, p.mondept,
(p.sal * 0.05) * coalesce(ns.nbsub, 0) as prime_enc
from distribill2._personnel p
left join distribill2._employe e on e.matr = p.matr
left join (
with recursive h as (
select sup, sub
from distribill2.dirige
union all

```



```

select h.sup, d.sub
from h
join distribill2.dirige d on d.sup = h.sub
)
select sup, count(*) as nbsub
from h
group by sup
) ns on ns.sup = p.matr;
```

```

13.5 Idée pédagogique importante

Les vues ne servent pas seulement à filtrer des données.

Elles servent aussi à :

- maintenir la compatibilité
- cacher une réorganisation interne du schéma
- exposer une représentation métier plus pratique

14. PL/pgSQL : programmer dans PostgreSQL

Le TD05 introduit PL/pgSQL.

14.1 Pourquoi PL/pgSQL ?

SQL pur est déclaratif et très puissant, mais parfois on a besoin :

- de variables
- de tests
- de boucles
- de gestion d'exceptions
- de fonctions réutilisables

14.2 Structure générale d'une fonction

```

```sql
create or replace function schema.nom_fonction(param type)
returns type
language plpgsql

```

```

as $$
declare
variable type;
begin
-- traitement
return variable;
end;
$$;
```

```

14.3 Fonction `anciennete`

Le TD demande une fonction qui calcule l'âge ou l'ancienneté.

Version robuste :

```

```sql
create or replace function distribill1.anciennete(dn date)
returns integer
language plpgsql
as $$
declare
age_entier integer;
begin
if dn > current_date then
raise exception 'Date incoherente : %', dn;
end if;

age_entier := extract(year from age(current_date, dn));
return age_entier;
end;
$$;
```

```

Utilisation :

```

```sql
select distribill1.anciennete(date '1984-06-06');
```

```

Application aux employés :

```
```sql
select matr, nome, distribill1.anciennete(dateemb::date) as anciennete
from distribill1.emp;
```
```

14.4 Fonction `nbdepts()`

```
```sql
create or replace function distribill1.nbdepts()
returns integer
language plpgsql
as $$
declare
n integer;
begin
select count(*) into n
from distribill1.dept;

if n = 0 then
raise exception 'Aucun departement trouve';
end if;

return n;
end;
$$;
```
```

Version paramétrée :

```
```sql
create or replace function distribill1.nbdepts(nom_recherche varchar)
returns integer
language plpgsql
as $$
declare
n integer;
begin
```

```

select count(*) into n
from distribill1.dept
where nomdept = nom_recherche;

return n;
end;
$$;
```

```

14.5 Fonction `propIng()`

Objectif :

- compter le total des employés
- compter les ingénieurs
- retourner un pourcentage

```

```sql
create or replace function distribill1.propIng()
returns numeric
language plpgsql
as $$
declare
nb_total integer;
nb_ing integer;
prop numeric;
begin
select count(*) into nb_total
from distribill1.emp;

select count(*) into nb_ing
from distribill1.emp
where poste = 'INGENIEUR';

prop := (nb_ing::numeric / nb_total::numeric) * 100;
return prop;

exception
when division_by_zero then
return null;
```

```

```
end;  
$$;  
``
```

14.6 Fonction `nbsub(matricule)`

Très intéressante car elle réutilise la récursivité.

```
``sql  
create or replace function distribill1.nbsub(p_matr integer)  
returns integer  
language plpgsql  
as $$  
declare  
n integer;  
existe integer;  
begin  
select count(*) into existe  
from distribill1.emp  
where matr = p_matr;  
  
if existe = 0 then  
raise exception 'Employe inexistant : %', p_matr;  
end if;  
  
with recursive h as (  
select sup, sub  
from distribill1.dirige  
where sup = p_matr  
  
union all  
  
select h.sup, d.sub  
from h  
join distribill1.dirige d on d.sup = h.sub  
)  
select count(*) into n  
from h;  
  
return n;
```

```
end;  
$$;  
...
```

14.7 Points clés PL/pgSQL à retenir

- `declare` : variables locales
- `select ... into` : affectation du résultat d'une requête
- `if ... then ... end if`
- `exception when ... then`
- `raise exception`

15. TD02 et TD03 : méthode complète de passage UML -> relationnel

15.1 Cas "Organisation des stages"

Classes :

- `CENTRES`
- `CLIENTS`
- `STAGES`
- `INSCRIPTIONS`

Lecture conceptuelle :

- un stage se déroule dans un centre
- un client peut s'inscrire à plusieurs stages
- un stage peut recevoir plusieurs clients
- `date\_inscription` appartient à l'association

Schéma type :

```
```sql  
create table centres(
 nom_centre varchar(50) primary key,
 capacite_centre integer,
 nom_pays varchar(50)
);

create table clients(
 nom_client varchar(50) primary key,
 adresse varchar(100),
 date_naissance date,
 date_inscription date
);
```

```

no_client integer primary key,
nom varchar(50) not null,
prenom varchar(50) not null,
adresse varchar(100),
code_postal varchar(10),
ville varchar(50),
tel_client varchar(20)
);

```

```

create table stages(
no_stage integer primary key,
niveau_stage varchar(30),
ou_centre varchar(50) not null references centres(nom_centre)
);

```

```

create table inscriptions(
no_client integer references clients(no_client),
no_stage integer references stages(no_stage),
date_inscription date not null,
primary key(no_client, no_stage)
);
```

```

15.2 Cas "Gestion de projets"

Classes :

- `FOURNISSEUR`
- `PROJET`
- `ARTICLE`
- `DOTATION`

`DOTATION` est une classe-association portant des attributs.

Traduction classique :

```

```sql
create table dotation(
no_fournisseur integer references fournisseur(no_fournisseur),
no_projet integer references projet(no_projet),

```

```
no_article integer references article(no_article),
quantite_dot integer not null,
nomenclature varchar(50),
primary key(no_fournisseur, no_projet, no_article)
);
...
```

### 15.3 Cas "Fleuve / affluent"

On a à la fois :

- une hiérarchie de spécialisation
- une relation réflexive ou proche d'une structure de graphe

Le point essentiel du TD est de bien distinguer :

- les identifiants
- les liens de composition / association
- les contraintes de rattachement

### 15.4 TD03 niveau 2 : contraintes métier plus fines

Le niveau 2 ajoute souvent :

- des associations ternaires
- des contraintes d'inclusion
- des dépendances entre associations

Exemple de l'exercice "société de services" :

- `possede(code\_comp, matricule\_emp)` : employé possède une compétence
- `intervenir(code\_comp, no\_affaire, matricule\_emp)` : employé intervient sur une affaire avec une compétence

Point très important :

`intervenir` doit rester cohérent avec `possede`.

Autrement dit :

si un employé intervient avec une compétence, il doit posséder cette compétence.

Le relationnel brut ne garantit pas toujours facilement ce genre de contrainte sans clé étrangère composite, contrainte avancée ou trigger.



### 15.5 Exemple de contrôle logique

Dans la valeur de base fournie, on voit :

```
```text
```

```
1000 M1 C2
```

```
```
```

or M1 possède :

- `C1`
- `C3`

mais pas `C2`.

Donc la contrainte métier n'est pas respectée.

### 15.6 Leçon importante

Toutes les contraintes UML ne se traduisent pas par de simples `primary key` et `foreign key`.

Certaines nécessitent :

- des contraintes plus riches
- des vues de contrôle
- des triggers
- ou des vérifications applicatives

## 16. Impact des choix de conception - TP4

Le TP4 est très important car il montre qu'un schéma n'est pas seulement "correct". Il doit aussi être pertinent.

### 16.1 Ce qu'il faut comparer

Quand plusieurs implantations sont possibles, on compare :

- nombre d'attributs
- nombre de n-uplets
- nombre de jointures nécessaires
- respect des cardinalités
- facilité à faire respecter l'intégrité

- coût en stockage
- coût en temps de requête

## 16.2 Table d'association ou clé étrangère directe ?

Question classique :

faut-il :

- mettre une clé étrangère dans une table
- ou créer une table d'association dédiée ?

##### On préfère une clé étrangère directe si :

- la relation est 1,N
- elle ne porte aucun attribut
- chaque élément du côté N n'est lié qu'à un seul élément du côté 1

##### On préfère une table d'association si :

- la relation est N,N
- la relation porte des attributs
- on veut garder l'historique ou plusieurs liens

## 16.3 Intégrité vs simplicité

Un schéma très "plat" peut sembler simple mais :

- duplique des données
- fragilise l'intégrité
- complique les mises à jour

Un schéma plus normalisé :

- demande plus de jointures
- mais protège mieux la cohérence

## 16.4 Le cas des inscriptions aux tournois d'échecs

Le script de peuplement du TP4 montre un scénario à fort volume :

- beaucoup de joueurs
- beaucoup de tournois
- une probabilité d'inscription

Cela fait exploser le nombre de lignes dans la table d'association `Inscription`.

Leçon :

- une bonne modélisation doit anticiper les volumes
- le coût de jointure devient réel
- la cardinalité des associations a un impact direct sur les performances

### **16.5 Ce que montrent les requêtes du TP4**

Les requêtes proposées testent plusieurs difficultés :

- filtre simple sur une table
- agrégation par ville
- jointure joueur / inscription / tournoi
- anti-jointure pour les joueurs non inscrits
- top-k avec sous-requêtes
- statistiques conditionnelles

### **16.6 Conclusion de conception**

Il n'existe pas une unique bonne traduction UML -> SQL.

Il existe plusieurs traductions, avec des compromis sur :

- lisibilité
- redondance
- intégrité
- performance
- évolutivité

## **17. Bonnes pratiques SQL et PostgreSQL**

### **17.1 Nommer proprement**

- contraintes nommées
- alias clairs
- tables qualifiées par le schéma si besoin

### **17.2 Utiliser les bons types**

- ``integer`` pour identifiants numériques
- ``numeric(10,2)`` pour montants
- ``date`` pour les dates
- ``varchar(n)`` pour les textes courts

### **17.3 Toujours tester les cas limites**

- table vide
- valeur absente
- département sans employé
- employé sans subalterne
- fournisseur sans fourniture

### **17.4 Penser en ensembles**

SQL ne raisonne pas ligne par ligne comme un langage impératif classique.

Il faut penser :

- sélection d'un ensemble
- transformation
- regroupement
- filtrage

### **17.5 Privilégier ``not exists`` à ``not in`` en présence potentielle de ``null``**

``not in`` peut avoir des comportements gênants si la sous-requête contient des ``null``.

### **17.6 Vérifier après chaque mise à jour**

Après un ``update`` ou un ``delete``, exécuter une requête de contrôle.

## **18. Erreurs classiques à éviter**

### **18.1 Oublier ``group by``**

Si tu mélanges colonnes agrégées et non agrégées, PostgreSQL proteste.

### **18.2 Confondre ``where`` et ``having``**

- ``where`` avant regroupement
- ``having`` après regroupement

### **18.3 Oublier les jointures explicites**

Les jointures implicites dans `where` fonctionnent, mais les `join ... on ...` sont plus lisibles et plus sûres.

### **18.4 Mettre du texte à la place d'un type structuré**

Exemple :

- stocker une date en `varchar` au lieu de `date`

Ça complique les comparaisons et les calculs.

### **18.5 Croire qu'une vue est une copie**

Une vue standard n'est pas une table recopiée. C'est une définition logique.

### **18.6 Croire qu'une clé étrangère suffit à exprimer toute règle métier**

Faux. Beaucoup de règles avancées ne sont pas capturées par les seules FK.

## **19. Fiche méthode pour examen / TD / TP**

### **19.1 Pour traduire UML -> relationnel**

1. repérer les classes
2. repérer les identifiants
3. repérer les cardinalités
4. traiter les associations 1,N
5. transformer les N,N en tables d'association
6. placer les attributs d'association
7. identifier les clés primaires
8. ajouter les clés étrangères
9. réfléchir aux contraintes métier non capturées automatiquement

### **19.2 Pour écrire une requête SQL**

1. lister les tables utiles
2. identifier les liens entre tables
3. écrire le `from` et les `join`
4. poser le `where`
5. ajouter `group by` si on agrège

6. ajouter `having` si on filtre des groupes
7. trier si besoin avec `order by`

### **19.3 Pour une requête difficile**

1. reformuler l'énoncé en français simple
2. reconnaître le motif :
  - existence
  - absence
  - totalité
  - comparaison au max
  - hiérarchie
3. découper en sous-problèmes
4. tester d'abord la sous-requête seule

### **19.4 Pour une fonction PL/pgSQL**

1. définir les paramètres
2. définir le type de retour
3. déclarer les variables
4. affecter avec `select ... into`
5. gérer les cas anormaux
6. retourner la valeur

## **20. Mini synthèse finale**

Si tu devais retenir l'essentiel de tous les documents, ce serait :

- le modèle relationnel sert à structurer proprement les données
- PostgreSQL permet d'implanter ce modèle avec des tables, clés et contraintes
- UML se traduit systématiquement en relations selon les cardinalités
- les requêtes SQL avancées se résolvent par une bonne lecture logique de l'énoncé
- les vues servent à simplifier, sécuriser et stabiliser l'accès aux données
- les CTE récursives sont la solution propre pour les hiérarchies
- PL/pgSQL permet d'ajouter de la logique procédurale et de la gestion d'exceptions
- les choix de conception ont un impact direct sur la performance, l'intégrité et la maintenabilité

## **21. Annexes - requêtes prêtes à réviser**

### 21.1 Tous les subalternes d'un employé

```
```sql
with recursive h as (
select sup, sub, 1 as niveau
from distribill1.dirige
where sup = 7566

union all

select h.sup, d.sub, h.niveau + 1
from h
join distribill1.dirige d on d.sup = h.sub
)
select *
from h;
```
```

### 21.2 Départements avec ou sans employés

```
```sql
select d.nodept, d.nomdept, count(e.matr) as effectif
from distribill1.dept d
left join distribill1.emp e on e.mondept = d.nodept
group by d.nodept, d.nomdept
order by d.nodept;
```
```

### 21.3 Fournisseurs fournissant tous les produits de Dijon

```
```sql
select f.nofour
from fournisseur f
where not exists (
select 1
from produit p
where p.origine = 'Dijon'
and not exists (
select 1
```

```
from fourniture fo
where fo.nofour = f.nofour
and fo.noproduit = p.noproduit
)
);
---
```

21.4 Vue avec contrôle

```
```sql
create or replace view LannionProducteurs as
select *
from Producteur
where ville = 'Lannion'
with check option;

```

## 21.5 Fonction de proportion des ingénieurs

```
```sql
select distribill1.propIng();
---
```

22. Conclusion

Tu as maintenant un support global qui relie tous les documents entre eux :

- la théorie relationnelle
- la traduction UML
- l'implantation SQL
- les requêtes simples et complexes
- la récursivité
- les vues
- PL/pgSQL
- l'analyse de conception

Le point vraiment important est de ne pas apprendre les requêtes "par coeur" sans comprendre le schéma et la logique des cardinalités. Une fois les relations, les clés et les dépendances bien comprises, les requêtes deviennent beaucoup plus naturelles.